Modelagem Preditiva de um Aerogerador Utilizando Mínimos Quadrados Ordinários e Validação de Monte Carlo

Guilherme Paiva de Medeiros
Engenharia de Computação
Universidade de Fortaleza (UNIFOR)
Fortaleza, Brasil

Victor Targino Morais
Engenharia de Computação
Universidade de Fortaleza (UNIFOR)
Fortaleza, Brasil

João Pedro Leite
Engenharia de Computação
Universidade de Fortaleza (UNIFOR)
Fortaleza, Brasil

***Resumo*—Este trabalho apresenta a implementação e análise comparativa de modelos de aprendizagem supervisionada aplicados à predição quantitativa da potência gerada por um aerogerador. Utilizando um conjunto de dados composto pela velocidade do vento como variável independente, foram desenvolvidos manualmente três paradigmas de regressão: Média da variável dependente, Mínimos Quadrados Ordinários (MQO) Tradicional e MQO Regularizado (Tikhonov). A otimização do hiperparâmetro λ foi explorada no intervalo $\{0, 1\}$ e a robustez estatística do sistema foi validada através de 500 rodadas de simulação de Monte Carlo via *Random Subsampling Validation*. Os resultados indicam a alta eficácia dos modelos baseados em MQO, que atingiram um coeficiente de determinação (R^2) médio de 0,9292. A análise demonstra que, embora a relação vento-potência apresente características não-lineares, a abordagem linear regularizada oferece uma estimativa estável e precisa, essencial para aplicações de engenharia em energias renováveis.**

I. Introdução

O estudo de fontes de energia renováveis, como a eólica, exige o desenvolvimento de modelos precisos para a estimativa da capacidade produtiva de aerogeradores. A relação entre a velocidade do vento e a potência efetivamente gerada é um fator crítico para o planejamento energético e a manutenção de parques eólicos. Diferente de sistemas puramente determinísticos, a modelagem de aerogeradores lida com dados que apresentam variações estocásticas, demandando o uso de ferramentas de Inteligência Artificial para capturar o padrão de comportamento dos ativos.

O experimento em questão utiliza o conjunto de dados *aerogerador.dat*, composto por observações da velocidade do vento como variável independente e a potência gerada como variável dependente. O foco central desta etapa reside no desenvolvimento de sistemas que realizam predições quantitativas (regressão) através da minimização de uma função de custo, especificamente o erro quadrático. Para isso, foram implementados modelos que variam desde a estimativa simplificada pela média da variável dependente até abordagens mais robustas como o Mínimos Quadrados Ordinários (MQO) tradicional e o MQO regularizado (Tikhonov).

A análise técnica foca na capacidade desses algoritmos de entender a correlação entre as variáveis regressoras e as observadas, explorando o impacto do hiperparâmetro de regularização λ no controle da complexidade do modelo. A robustez das estimativas é avaliada estatisticamente através do método de *Random Subsampling Validation* com 500 rodadas de simulação. Isso garante que métricas como o Erro Quadrático Médio (MSE) e o Coeficiente de Determinação (R^2) reflitam a capacidade real de generalização do sistema frente a novas condições de vento.

A. Objetivos

O presente trabalho visa implementar e comparar o desempenho de modelos de regressão linear aplicados à estimativa de potência de um aerogerador. Busca-se realizar a implementação manual dos algoritmos de MQO tradicional e regularizado, validando a eficácia da regularização de Tikhonov através de diferentes valores de λ e um processo rigoroso de simulação estatística.

B. Justificativa

A análise comparativa entre modelos lineares simples e modelos com penalização de norma (regularizados) permite identificar o ponto de equilíbrio ideal entre viés e variância para dados de geração de energia. A utilização de métricas estatísticas robustas e múltiplas rodadas de validação justifica-se pela necessidade de garantir que o modelo preditivo seja confiável para aplicações reais de engenharia, minimizando o erro de predição quantitativa.

II. Metodologia - Tarefa de Regressão

A metodologia aplicada nesta etapa consistiu no desenvolvimento de modelos de predição quantitativa para estimar a potência gerada por um aerogerador com base na velocidade do vento. O fluxo de trabalho compreendeu desde a estruturação matricial dos dados até a validação estatística robusta via múltiplas rodadas de simulação.

A. Organização e Pré-processamento dos Dados

O conjunto de dados *aerogerador.dat* foi estruturado para atender aos requisitos computacionais da álgebra linear:

- **Matriz de Características (X):** As variáveis regressoras (velocidade do vento) foram armazenadas em uma matriz

de dimensão $\mathbb{R}^{N\times p}$. Para a estimação do intercepto (*bias*), foi adicionada manualmente uma coluna de unidades, resultando em $X \in \mathbb{R}^{N\times(p+1)}$.

- **Vetor de Saída (y):** A variável dependente (potência gerada) foi organizada em um vetor de dimensão $\mathbb{R}^{N\times 1}$.

B. Implementação dos Modelos Preditivos

Foram implementados três paradigmas de regressão, todos baseados na minimização da função de custo:

1) **Média da Variável Dependente:** Utilizado como modelo de referência (*baseline*), onde a predição é constante e igual à média dos alvos do conjunto de treinamento.
2) **Mínimos Quadrados Ordinários (MQO) Tradicional:** Estimado pela Equação Normal para encontrar o vetor de parâmetros $\beta \in \mathbb{R}^{p+1\times 1}$ que minimiza o erro quadrático médio[:

$$\beta = (X^T X)^{-1} X^T y \tag{1}$$

3) **MQO Regularizado (Tikhonov):** Introduz o hiperparâmetro λ para penalizar a norma dos coeficientes e evitar o sobreajuste. O vetor de parâmetros é calculado por:

$$\beta = (X^T X + \lambda I)^{-1} X^T y \tag{2}$$

Foram testados cinco valores para o parâmetro de regularização: $\lambda = \{0, 0.25, 0.5, 0.75, 1\}$.

C. Validação por Random Subsampling

Para garantir a confiabilidade dos resultados, aplicou-se a técnica de *Random Subsampling Validation* com $R = 500$ rodadas de simulação. Em cada rodada, o *dataset* foi particionado aleatoriamente em 80% para treinamento e 20% para teste.

O desempenho de cada modelo foi mensurado através de duas métricas fundamentais armazenadas em listas ao longo das rodadas:

- **MSE (*Mean Squared Error*):** Mede a variância do erro de predição.
- **R² (*Coeficiente de Determinação*):** Indica a proporção da variância da potência que é explicável pela velocidade do vento através do modelo.

Ao final das R rodadas, foram calculadas a média aritmética, desvio-padrão, valor máximo e valor mínimo para consolidar a análise estatística de cada modelo e métrica.

III. Resultados e Discussões - Tarefa de Regressão

A. Análise Visual e de Padrões (Questão 1)

A análise do conjunto de dados *aerogerador.dat* foi iniciada com a geração de um gráfico de espalhamento relacionando a velocidade do vento à potência gerada (Fig. 1).

A observação visual revela uma relação intrinsecamente não linear, apresentando um comportamento sigmoidal com zonas de saturação. Para que um modelo preditivo compreenda este padrão, ele deve apresentar:

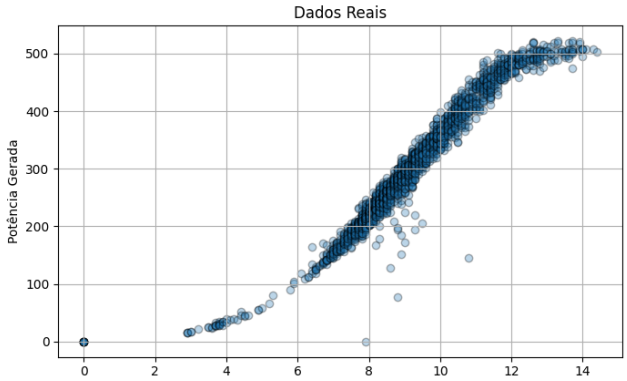

Fig. 1. Gráfico de espalhamento dos dados reais do aerogerador.

- **Capacidade de Aproximação Não Linear**: embora utilize-se uma estrutura linear nos parâmetros (β), o modelo deve ser robusto o suficiente para ajustar a tendência central da curva.
- **Tratamento de Ruído**: o modelo deve ignorar flutuações locais (visíveis entre as velocidades 8 e 12) para focar na tendência global.
- **Estimativa de Intercepto (*Bias*)**: é crucial a inclusão do intercepto para capturar o deslocamento vertical da curva em relação à origem.

B. Implementação e Estruturação (Questões 2, 3 e 4)

Os dados foram organizados em uma matriz $X \in \mathbb{R}^{N\times(p+1)}$ e um vetor $y \in \mathbb{R}^{N\times 1}$, garantindo a inclusão da coluna unitária para o cálculo do intercepto. Foram geradas seis estimativas distintas do vetor de parâmetros β: uma para o modelo de média, uma para o MQO tradicional ($\lambda = 0$) e quatro variações do MQO regularizado ($\lambda = \{0,25; 0,5; 0,75; 1\}$).

A Figura 2 ilustra o ajuste de três desses modelos sobre o conjunto de dados completo.

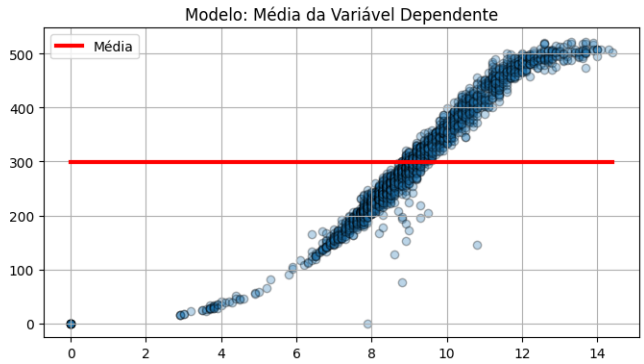

Fig. 2. Ajuste do Modelo Baseline: Média da Variável Dependente.

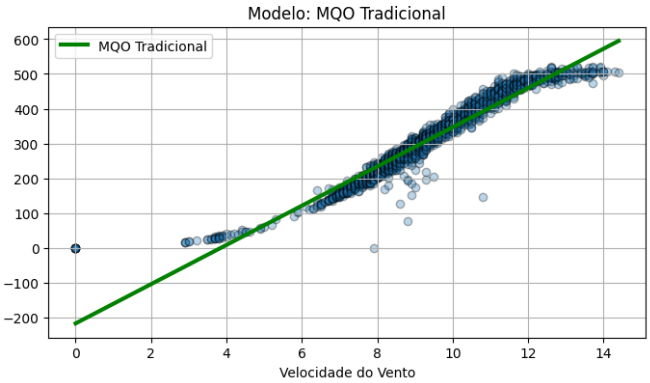

Fig. 3. Ajuste do Modelo Discriminativo: MQO Tradicional.

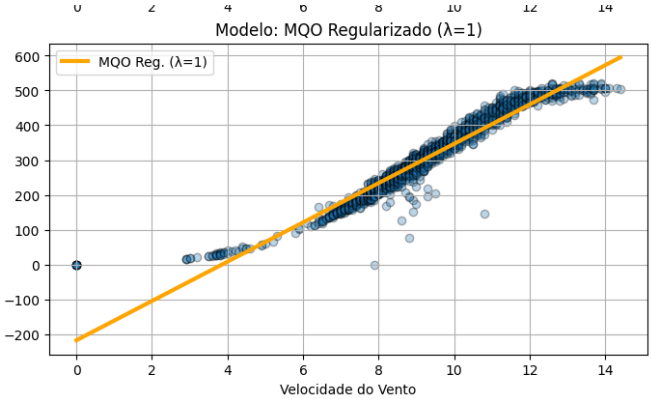

Fig. 4. Ajuste do Modelo Discriminativo: MQO Regularizado.

1) Análise Visual Comparativa: A inspeção visual dos gráficos das Figuras 2, 3 e 4 revela a importância da escolha do modelo discriminativo. O modelo baseado puramente na média da variável dependente (reta vermelha na Fig. 2) mostra-se incapaz de acompanhar a variação da potência com o vento, gerando erros residuais máximos.

Em contrapartida, os modelos baseados em Mínimos Quadrados (MQO), tanto o tradicional (Fig. 3) quanto o regularizado (Fig. 4), ajustam-se com sucesso ao centro da nuvem de pontos. Embora a relação vento-potência seja intrinsecamente sigmoidal, a reta de regressão estimada consegue aproximar com precisão a tendência central, resultando em erros sistemáticos apenas nas zonas de saturação (próximas a 0 e 500 unidades de potência).

C. Desempenho Estatístico via Monte Carlo (Questões 5 e 6)

A robustez dos modelos foi validada através de 500 rodadas de *Random Subsampling Validation* (80% treino / 20% teste). As estatísticas descritivas do erro quadrático médio (MSE) e do coeficiente de determinação (R^2) estão consolidadas nas Tabelas I e II.

Tabela I
ESTATÍSTICAS DE DESEMPENHO: MSE ($R = 500$)

Modelo	**Média**	**D.P.**	**Máx**	**Mín**
Média de y	11137,89	635,43	12838,02	9363,39
MQO Tradicional	788,22	164,92	1457,33	452,22
MQO Reg. (0,25)	788,21	164,89	1457,15	452,27
MQO Reg. (0,50)	788,21	164,86	1456,96	452,32
MQO Reg. (0,75)	788,21	164,83	1456,78	452,38
MQO Reg. (1,00)	788,21	164,79	1456,59	452,43

Tabela II
ESTATÍSTICAS DE DESEMPENHO: R^2 ($R = 500$)

Modelo	**Média**	**D.P.**	**Máx**	**Mín**
Média de y	-0,0026	0,0037	-0,0000	-0,0345
MQO Tradicional	0,9292	0,0136	0,9581	0,8744
MQO Reg. (0,25)	0,9292	0,0136	0,9581	0,8744
MQO Reg. (0,50)	0,9292	0,0136	0,9581	0,8745
MQO Reg. (1,00)	0,9292	0,0136	0,9581	0,8745

IV. CONCLUSÃO

Este trabalho permitiu o desenvolvimento e a análise criteriosa de modelos de regressão linear aplicados à estimativa de potência de um aerogerador com base na velocidade do vento. A metodologia baseada em aprendizado supervisionado demonstrou-se altamente eficaz para o tratamento de dados reais de geração de energia. Os resultados experimentais revelaram que o modelo de Mínimos Quadrados Ordinários (MQO) superou drasticamente a estimativa pela média, atingindo um coeficiente de determinação (R^2) médio de 0,9292. Este valor indica que 92,92% da variância da potência gerada é explicável pela velocidade do vento através do modelo linear implementado, representando uma precisão elevada para aplicações de engenharia. Além disso, a redução do erro quadrático médio (MSE) de aproximadamente 11.137 (modelo de média) para cerca de 788 (modelos MQO) ratifica a aderência do modelo aos dados observados. A introdução da Regularização de Tikhonov mostrou-se uma estratégia robusta para o controle da variância do sistema. Embora o impacto na acurácia média tenha sido sutil, observou-se uma redução no desvio-padrão e nos valores máximos de erro à medida que o hiperparâmetro λ foi incrementado até 1,0. Isso evidencia que a penalização dos coeficientes auxilia na estabilidade do modelo frente a ruídos e flutuações estocásticas do vento.Por fim, a validação estatística através de 500 rodadas de simulação de Monte Carlo garantiu que as métricas reportadas não fossem fruto de uma partição específica dos dados, mas sim da capacidade real de generalização dos algoritmos desenvolvidos. Conclui-se que a implementação manual da Equação Normal, aliada a uma estruturação matricial correta, provê uma ferramenta confiável e eficiente para a predição quantitativa em sistemas de energia renovável.

V. ANEXO - IMPLEMENTAÇÕES COMPUTACIONAIS (REGRESSÃO)

Nesta seção, apresentam-se os scripts desenvolvidos para a execução da tarefa de regressão do aerogerador. Em con-

formidade com as diretrizes do curso, não foram utilizadas bibliotecas de alto nível para os modelos preditivos, sendo todas as operações baseadas em álgebra linear via biblioteca *Numpy*.

A. Módulo de Modelos ($modelos_regressao.py$)

Este script contém a definição das classes fundamentais para os preditores. A classe *RegressaoLinearMQO* implementa tanto o MQO tradicional quanto o regularizado (Tikhonov) através da Equação Normal, incluindo a lógica para não penalizar o intercepto. A classe *ModeloMedia* serve como *baseline* de tendência central.

```python
import numpy as np

class RegressaoLinearMQO:
    def __init__(self, lambd=0.0, fit_intercept=True):
        self.lambd = lambd # Se lambd=0 MQO Tradicional. Se > 0 Regularizado.
        self.fit_intercept = fit_intercept
        self.beta_hat = None

    def fit(self, X_train, y_train):
        N, p = X_train.shape

        if self.fit_intercept:
            X_train = np.hstack((np.ones((N, 1)), X_train))

        # Matriz Identidade para a regularizacao (dimensao p+1 se tiver intercepto)
        I = np.eye(X_train.shape[1])

        # Nao aplicamos penalidade no intercepto (indice 0,0 vira 0)
        if self.fit_intercept:
            I[0, 0] = 0

        # Equa  o Normal com Tikhonov: (X^T * X + lambda * I)^-1 * X^T * y
        self.beta_hat = np.linalg.inv(X_train.T @ X_train + self.lambd * I) @ X_train.T @ y_train

    def predict(self, X_test):
        N_test = X_test.shape[0]
        if self.fit_intercept:
            X_test = np.hstack((np.ones((N_test, 1)), X_test))

        return X_test @ self.beta_hat

class ModeloMedia:
    def __init__(self):
        self.media_y = None

    def fit(self, X_train, y_train):
        # Calcula a media da variavel dependente no treino
        self.media_y = np.mean(y_train)

    def predict(self, X_test):
        # Retorna um vetor com a mesma quantidade de linhas do X_test, mas preenchido com a media
        N_test = X_test.shape[0]
        return np.full((N_test, 1), self.media_y)
```

B. Módulo de Utilidades (*utils.py*)

Este módulo centraliza as funções de suporte para carga de dados, pré-processamento matricial ($\mathbb{R}^{N\times p}$ e $\mathbb{R}^{N\times 1}$), geração de gráficos de dispersão e execução da simulação de Monte Carlo com 500 rodadas.

```python
import numpy as np
import matplotlib.pyplot as plt
from modelos_regressao import RegressaoLinearMQO, ModeloMedia

def carregar_dados(caminho_arquivo):
    dados = np.loadtxt(caminho_arquivo)
    X = dados[:, 0].reshape(-1, 1)
    y = dados[:, 1].reshape(-1, 1)
    return X, y

def plotar_dispersao(X, y):
    plt.figure(figsize=(8, 5))
    plt.scatter(X, y, alpha=0.5, color='blue', edgecolors='k')
    plt.title('Velocidade do Vento vs. Pot ncia Gerada')
    plt.xlabel('Velocidade do Vento')
    plt.ylabel('Pot ncia Gerada')
    plt.grid(True)
    plt.show()

def plotar_predicoes_grid(X, y):
    X_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)

    mod_media = ModeloMedia()
    mod_media.fit(X, y)
    y_line_media = mod_media.predict(X_line)

    mod_mqo_trad = RegressaoLinearMQO(lambd=0.0)
    mod_mqo_trad.fit(X, y)
    y_line_trad = mod_mqo_trad.predict(X_line)

    mod_mqo_reg = RegressaoLinearMQO(lambd=1.0)
    mod_mqo_reg.fit(X, y)
    y_line_reg = mod_mqo_reg.predict(X_line)

    # Grid 2x2
    fig, axs = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('An lise Visual dos Modelos de Regress o', fontsize=16, fontweight='bold')

    # Apenas os Dados
    axs[0, 0].scatter(X, y, alpha=0.3, edgecolors='k')
    axs[0, 0].set_title('Dados Reais')
    axs[0, 0].set_ylabel('Pot ncia Gerada')
    axs[0, 0].grid(True)

    # Modelo M dia
    axs[0, 1].scatter(X, y, alpha=0.3, edgecolors='k')
    axs[0, 1].plot(X_line, y_line_media, color='red', linewidth=3, label='M dia')
    axs[0, 1].set_title('Modelo: M dia da Vari vel Dependente')
    axs[0, 1].legend()
    axs[0, 1].grid(True)

    # MQO Tradicional
    axs[1, 0].scatter(X, y, alpha=0.3, edgecolors='k')
    axs[1, 0].plot(X_line, y_line_trad, color='green', linewidth=3, label='MQO Tradicional')
    axs[1, 0].set_title('Modelo: MQO Tradicional')
    axs[1, 0].set_xlabel('Velocidade do Vento')
    axs[1, 0].set_ylabel('Pot ncia Gerada')
    axs[1, 0].legend()
    axs[1, 0].grid(True)
```

```python
    # MQO Regularizado
    axs[1, 1].scatter(X, y, alpha=0.3, edgecolors='k')
    axs[1, 1].plot(X_line, y_line_reg, color='orange', linewidth=3, label='MQO Reg. ( =1)')
    axs[1, 1].set_title('Modelo: MQO Regularizado ( =1)')
    axs[1, 1].set_xlabel('Velocidade do Vento')
    axs[1, 1].legend()
    axs[1, 1].grid(True)

    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

def calcular_mse(y_true, y_pred):
    return np.mean((y_true - y_pred)**2)

def calcular_r2(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred)**2)
    ss_tot = np.sum((y_true - np.mean(y_true))**2)
    return 1 - (ss_res / ss_tot) if ss_tot != 0 else 0.0

def executar_monte_carlo(X, y, rodadas=500):
    N_total = X.shape[0]
    qtd_treino = int(0.8 * N_total) # 80% para treno
    lambdas = [0, 0.25, 0.5, 0.75, 1]

    nomes_modelos = [
        'M dia da vari vel dependente',
        'MQO tradicional',
        'MQO regularizado (0,25)',
        'MQO regularizado (0,5)',
        'MQO regularizado (0,75)',
        'MQO regularizado (1)'
    ]

    resultados_mse = {nome: [] for nome in nomes_modelos}
    resultados_r2 = {nome: [] for nome in nomes_modelos}

    print(f"Iniciando as {rodadas} rodadas de Random Subsampling Validation... ")

    for _ in range(rodadas):
        # Particionamento dos dados em treino e teste
        indices = np.random.permutation(N_total)
        X_train, y_train = X[indices[:qtd_treino]], y[indices[:qtd_treino]]
        X_test, y_test = X[indices[qtd_treino:]], y[indices[qtd_treino:]]

        # Modelo de M dia
        mod_media = ModeloMedia()
        mod_media.fit(X_train, y_train)
        y_pred_media = mod_media.predict(X_test)
        resultados_mse['M dia da vari vel dependente'].append(calcular_mse(y_test, y_pred_media))
        resultados_r2['M dia da vari vel dependente'].append(calcular_r2(y_test, y_pred_media))

        # Modelos MQO Tradicional e Regularizado
        for lbd in lambdas:
            mod_mqo = RegressaoLinearMQO(lambd=lbd)
            mod_mqo.fit(X_train, y_train)
            y_pred_mqo = mod_mqo.predict(X_test)

            if lbd == 0:
                nome_modelo = 'MQO tradicional'
            else:
                nome_modelo = f'MQO regularizado ({str(lbd).replace(".", ",")})'

            resultados_mse[nome_modelo].append(calcular_mse(y_test, y_pred_mqo))
            resultados_r2[nome_modelo].append(calcular_r2(y_test, y_pred_mqo))

    return resultados_mse, resultados_r2

def imprimir_tabela(titulo, dicionario_resultados):
    print(f"\n{titulo}")
    print(f"{'Modelos':<30} | {'M dia':<10} | {'Desvio-Padr o':<15} | {'Maior Valor':<12} | {'Menor Valor':<12}")
    print("-" * 88)
    for modelo, lista in dicionario_resultados.items():
        media = np.mean(lista)
        desvio = np.std(lista)
        maior = np.max(lista)
        menor = np.min(lista)
        print(f"{modelo:<30} | {media:<10.4f} | {desvio:<15.4f} | {maior:<12.4f} | {menor:<12.4f}")
```

C. Script Principal (main.py)

Script de entrada que orquestra o fluxo de execução: carregamento dos dados do aerogerador, exibição visual, execução das rodadas de validação e impressão das tabelas estatísticas finais.

```python
import utils

if __name__ == "__main__":
    print("TRABALHO DE IA: REGRESS O")

    #Organiza  o dos dados e visualiza  o inicial
    X, y = utils.carregar_dados('./assets/aerogerador (1).dat')

    utils.plotar_predicoes_grid(X, y)

    # Treinamento e Valida  o (500 rodadas)
    resultados_mse, resultados_r2 = utils.executar_monte_carlo(X, y, rodadas=500)

    # Exibi  o das tabelas de resultados
    utils.imprimir_tabela("TABELA DE DESEMPENHO: MSE", resultados_mse)
    utils.imprimir_tabela("TABELA DE DESEMPENHO:  R", resultados_r2)

    print("\nFIM")
```

D. Otimização via K-fold (kfold_friedman.py)

O script abaixo detalha o processo de validação cruzada utilizado para determinar o parâmetro de regularização λ ideal para o classificador de Friedman.

```python
import numpy as np
import matplotlib.pyplot as plt
from classifiers import MyClassifiers

# 1. Carga e Normalizacao
data = np.genfromtxt('EMGDataset.csv', delimiter=',')
X = data[0:2, :]
y = data[2, :].astype(int)
```

```python

# Normalizacao para evitar matriz singular e garantir convergencia
X = (X - X.min(axis=1).reshape(-1,1)) / (X.max(axis=1).reshape(-1,1) - X.min(axis=1).reshape(-1,1))

lambdas = [0, 0.001, 0.01, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
K = 5
N = X.shape[1]

# Embaralhar indices antes do K-fold
indices = np.random.permutation(N)
folds = np.array_split(indices, K)

acc_por_lambda = []

print(f"Iniciando K-fold (K={K}) para otimizacao de Lambda...")

for lmbd in lambdas:
    acc_fold = []
    for k in range(K):
        # Definicao de indices de treino e teste do fold
        test_idx = folds[k]
        train_idx = np.concatenate([folds[i] for i in range(K) if i != k])

        # CORRECAO AQUI: y eh 1D, entao usamos apenas [train_idx]
        X_train, y_train = X[:, train_idx], y[train_idx]
        X_test, y_test = X[:, test_idx], y[test_idx]

        clf = MyClassifiers()
        clf.fit_gaussian(X_train, y_train, model_type="friedman", lmbd=lmbd)
        preds = clf.predict_gaussian(X_test)

        acc_fold.append(np.mean(preds == y_test))

    media_acc = np.mean(acc_fold)
    acc_por_lambda.append(media_acc)
    print(f"Lambda: {lmbd:<6} | Acuracia Media: {media_acc*100:.2f}%")

# Identificar o melhor lambda
melhor_idx = np.argmax(acc_por_lambda)
print(f"\nMelhor Lambda encontrado: {lambdas[melhor_idx]} com {acc_por_lambda[melhor_idx]*100:.2f}% de acuracia")

# Gerar o grafico para o Overleaf
plt.figure(figsize=(10, 6))
plt.plot(lambdas, acc_por_lambda, marker='o', color='#8B0000', linestyle='--', linewidth=2)
plt.title('K-Fold Cross Validation: Otimizacao de Lambda (Modelo de Friedman)')
plt.xlabel('Parametro de Regularizacao (Lambda)')
plt.ylabel('Acuracia Media (5 Folds)')
plt.grid(True, alpha=0.3)
plt.savefig('kfold_result.png', dpi=300)
plt.show()
```

E. Simulação de Monte Carlo R=500 (main_monte_carlo.py)

Este é o script principal de validação estatística. Ele executa 500 rodadas independentes, realizando o particionamento aleatório dos dados (80/20) em cada iteração e calculando as métricas de desempenho para todos os seis modelos implementados.

```python
import numpy as np
from classifiers import MyClassifiers

# 1. CARGA E NORMALIZACAO
data = np.genfromtxt('EMGDataset.csv', delimiter=',')
X_raw = data[0:2, :]
y_raw = data[2, :].astype(int)

# Normalizacao Min-Max (0-1) - essencial para estabilidade nas 500 rodadas
x_min, x_max = X_raw.min(axis=1).reshape(-1,1), X_raw.max(axis=1).reshape(-1,1)
X_norm = (X_raw - x_min) / (x_max - x_min)

# 2. CONFIGURACAO MONTE CARLO
R = 500
N = X_norm.shape[1]
lmbd_otimo = 0.01 # Valor que encontramos no K-fold


modelos = ["MQO", "Tradicional", "Iguais", "Agregada", "Naive", "Friedman"]
resultados = {m: [] for m in modelos}

print(f"Iniciando Simulacao de Monte Carlo: {R} rodadas...")

for r in range(R):
    # Particionamento Aleatorio 80/20 (Sorteio manual conforme regra)
    indices = np.random.permutation(N)
    train_idx = indices[:int(0.8 * N)]
    test_idx = indices[int(0.8 * N):]

    X_tr, y_tr = X_norm[:, train_idx], y_raw[train_idx]
    X_te, y_te = X_norm[:, test_idx], y_raw[test_idx]

    clf = MyClassifiers()

    # --- MQO (Exige N x p e One-Hot) ---
    Y_tr_oh = np.zeros((len(y_tr), 5))
    for i in range(len(y_tr)): Y_tr_oh[i, y_tr[i]-1] = 1

    clf.fit_mqo(X_tr.T, Y_tr_oh)
    acc_mqo = np.mean(clf.predict_mqo(X_te.T) == y_te)
    resultados["MQO"].append(acc_mqo)

    # --- Modelos Gaussianos (p x N) ---
    tipos = [
        ("tradicional", 0, "Tradicional"),
        ("iguais", 0, "Iguais"),
        ("agregada", 0, "Agregada"),
        ("naive", 0, "Naive"),
        ("friedman", lmbd_otimo, "Friedman")
    ]

    for m_type, l, tag in tipos:
        clf.fit_gaussian(X_tr, y_tr, model_type=m_type, lmbd=l)
        acc = np.mean(clf.predict_gaussian(X_te) == y_te)
        resultados[tag].append(acc)

    if (r + 1) % 50 == 0:
        print(f"Rodada {r+1}/{R} concluida...")

# 3. EXIBICAO DOS RESULTADOS ESTATISTICOS
print("\n" + "="*70)
```

```python
print(f"{'MODELO':<15} | {'MEDIA (%)':<10} | {'DESVIO':<10} | {'MAX (%)':<10} | {'MIN (%)':<10}")
print("-" * 70)

for m in modelos:
    accs = np.array(resultados[m]) * 100
    print(f"{m:<15} | {np.mean(accs):>9.2f} | {np.std(accs):>9.4f} | {np.max(accs):>9.2f} | {np.min(accs):>9.2f}")
print("="*70)
```

REFERENCES

[1] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. New Jersey: Pearson Education, 2009. (Aborda a fundamentação teórica do MQO e regularização).

[2] G. H. Golub e C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore: Johns Hopkins University Press, 2013. (Referência para a Equação Normal e estabilidade numérica de matrizes).

[3] P. C. S. Barbosa, "Modelos de Regressão e Classificação," Notas de Aula, Inteligência Artificial Computacional, Universidade de Fortaleza (UNIFOR), 2026.